

Python Caesar Cipher

D. Leeuw

7 oktober 2026

1.0.0

© 2025-2026 Dennis Leeuw



Dit werk is uitgegeven onder de Creative Commons BY-NC-SA Licentie en laat anderen toe het werk te kopiëren, distribueren, vertonen, op te voeren, en om afgeleid materiaal te maken, zolang de auteurs en uitgever worden vermeld als maker van het werk, het werk niet commercieel gebruikt wordt en afgeleide werken onder identieke voorwaarden worden verspreid.

1 Over dit Document

1.1 Leerdoelen

1. Het doel is om vertrouwd te raken met de built in functies van Python

1.2 Voorkennis

De lezer die deze opdrachten wil maken dient te weten

- hoe if/else/elif wordt gebruikt in Python
- hoe for/while wordt gebruikt in Python met gebruik van break en continue
- hoe builtin functies in Python worden aangesproken en waar er documentatie over gevonden kan worden

2 Caesar Cipher

Julius Caesar gebruikte encryptie om zijn orders door te geven aan zijn ondergeschikten. De reden dat hij encryptie gebruikte was om te zorgen dat zijn orders geheim bleven. Als de persoon die het bericht moest overbrengen gevangen werd genomen of werd gedood dan had de vijand alleen het ge-encrypte bericht en kende dus niet meteen de opdracht die uitgevoerd moest worden.

De techniek die Julius Caesar gebruikte was dat hij de letters van zijn bericht 1 of meer posities op schoof in het alfabet, dus een a werd dan bijvoorbeeld een d en een b werd dan een e. Door dit op een vaste manier te doen, konden de ontvangers het bericht relatief makkelijk decoderen, terwijl die vijand die niet wist hoe vaak een letter verschoven was, als ze al wisten dat het om een verschuiving ging, niets met het bericht konden. In de tijd dat Julius Caesar deze techniek gebruikte, ongeveer 58 voor Christus, waren er sowieso niet veel mensen die konden lezen.

Stel dat we een bericht hebben met de tekst: 'Dit is een geheim bericht' en we schuiven alle letters één plaats op in het alfabet dan wordt het ge-encrypte bericht 'Eju jt ffo hfjn cfsjdiu'. Dat is al behoorlijk onleesbaar. Als we ook nog de spaties verwijderen 'Ejutffohfjncfsjdiu' dan is het totaal onleesbaar geworden.

Later werd deze techniek bekend onder de naam: Caesar Cipher

Extra informatie over het Caesar Cipher:

- https://www.splunk.com/en_us/blog/learn/caesar-cipher.html
- <https://nl.wikipedia.org/wiki/Caesarcijfer>

2.1 Opdracht

Beschrijf in een flowdiagram hoe je de verschuiving zou kunnen uitvoeren. Hou daarbij rekening met de volgende zaken:

- Gebruik een variabele voor het aantal keer dat er geschoven moet worden in het alfabet
- Geef deze variabele een waarde tussen 1 en 9 om de code niet te complex te maken
- Denk erom dat je na de Z weer bij A moet beginnen, na z weer bij a en na 9 komt weer 0.
- Overige tekens (leestekens) veranderen niet.

Maak een script dat een substitutie uitvoert (encryptie). Een variable bevat het te encrypten bericht en de output van het script is het encrypte bericht.

2.2 Tip: ASCII-tabel

Computers zijn rekenaars, ze werken alleen met getallen. Mensen werken graag met tekst. Om een computer om te laten gaan met tekst moet tekst voor de computer omgezet worden in getallen. Als elk programma andere getallen gebruikt voor deze omzetting, dan wordt het een zooitje. We hebben een standaard vertaal tabel nodig om tekst om te zetten in getallen. Eén van deze tabellen is de ASCII-tabel (zie tabel 1). ASCII staat voor American Standard Code for Information Interchange en dat is precies wat het doet. Het is een standaard om informatie uitwisseling te vergemakkelijken.

Om characters om te zetten naar een binaire code nemen we een character, zoeken deze op in de tabel. De kolom geeft de eerste 3 bits en de rij geeft de tweede 4 bits, zo komen we op 7-bits:

A 100 0001 = 65

a 110 0001 = 97

0 011 0000 = 48+0 = 48

9 011 1001 = 48+9 = 57

Het alfabet is op volgorde, dus B is 66 en b is 98 etc.

Tabel 1: ASCII-table

	000	001	010	011	100	101	110	111
...0000	NUL	DLE	SPC	0	@	P	'	p
...0001	SOH	DC1	!	1	A	Q	a	q
...0010	STX	DC2	"	2	B	R	b	r
...0011	ETX	DC3	#	3	C	S	c	s
...0100	EOT	DC4	\$	4	D	T	d	t
...0101	ENQ	NAK	%	5	E	U	e	u
...0110	ACK	SYN	&	6	F	V	f	v
...0111	BEL	ETB	,	7	G	W	g	w
...1000	BS	CAN	(	8	H	X	h	x
...1001	HT	EM	)	9	I	Y	i	y
...1010	LF	SUB	*	:	J	Z	j	z
...1011	VT	ESC	+	;	K		k	{
...1100	FF	FS	,	<	L	\	l	
...1101	CR	GS	-	=	M	]	m	}
...1110	SO	RS	.	>	N	^	n	~
...1111	SI	US	/	?	O	_	o	DEL

2.3 Andere vertaaltabellen

De ASCII-tabel is één van de oudste en was een van de meest gebruikte conversie-tabellen, zolang als computers voornamelijk door Engelstaligen werden gebruikt. Met de karakters van bijvoorbeeld de Europese of Slavische talen werd het allemaal wat lastiger en was het niet mogelijk om al deze karakters te vangen in 7-bits. Er ontstonden dan ook nadere conversie tabellen zoals UTF (Unicode Transformation Format) in 8-bits, 16-bits of zelfs 32-bits. Andere standaarden zijn de ISO 8859-1 en de Windows-1251 (ANSI Latin 1 of ANSI) standaard. Als we ook de Aziatische karakters willen kunnen weergeven dan redder we het ook niet met 8-bits, daar hebben we de 16 en 32-bits varianten voor.

2.4 Tip: Python conversie functies

Python heeft builtin functies die nummers kunnen omzetten naar characters en characters naar nummers. Deze functies kan je vinden op o.a. de site van W3Schools: https://www.w3schools.com/python/python_ref_functions.asp of op de site van Python (<https://docs.python.org/3/library/functions.html>). Zoek de functie op die je nodig hebt om een character om te zetten naar een number en de functie die het omgekeerde doet. Test de

functies in een script.

We hebben gezien dat de reeks nummers voor uppercase letters een aparte reeks is van de reeks met lowercase letters. Daar moeten we rekening mee houden. Ook als we bijvoorbeeld de letter z één plaats opschuiven dan moet het de a worden en niet { zoals dat volgens de ASCII-tabel zou worden. Om te weten te komen of een variabele uppercase of lowercase is kunnen we `islower` en `isupper` gebruiken. Voor het detecteren of een variabele een cijfer is is er `isnumeric`. Dat ziet er in Python code zo uit:

```
chars = [ 'a', 'A', '1', 'B', 'b', '2' ]
for char in chars:
    if char.islower():
        print(f"{char} is lowercase")
    elif char.isupper():
        print(f"{char} is uppercase")
    elif char.isnumeric():
        print(f"{char} is numeric")
```

Extra informatie over uppercase, lowercase en numeric detectie:

- https://www.w3schools.com/python/pythn_ref_string.asp
- <https://docs.python.org/3/library/stdtypes.html#string-methods>